

Évaluation NoSQL — Diagnostic, choix d'architecture et mini-application

Contexte général

Vous travaillez pour une petite agence spécialisée dans la conception d'applications web exploitant des données variées : utilisateurs, votes, historiques, relations entre entités, classements, événements en temps réel, collections de contenus, etc.

L'agence souhaite développer plusieurs prototypes autour de deux univers :

- la **Coupe du monde de football** ;
- l'univers **Pokémon**.

Votre mission n'est pas seulement de développer une application. Vous devez d'abord agir comme une équipe de consultants NoSQL : analyser les besoins fonctionnels, identifier les types de données manipulées, puis choisir les modèles NoSQL les plus adaptés.

L'objectif est de montrer que vous savez faire la différence entre les principaux modèles NoSQL :

- **Document** ;
- **Clé-valeur** ;
- **Colonne** ;
- **Graphe**.

Vous devrez ensuite choisir une application à développer en groupe et justifier vos choix techniques.

Objectifs de l'évaluation

À la fin de cette évaluation, vous devrez être capables de :

1. analyser un besoin applicatif simple ;
2. identifier les types de données à manipuler ;
3. choisir les modèles NoSQL les plus adaptés ;
4. justifier les choix techniques réalisés ;
5. développer une mini-application utilisant au minimum deux modèles NoSQL ;
6. présenter clairement votre architecture et vos choix.

Organisation du travail

L'évaluation se déroule en deux grandes étapes.

Étape 1 — Diagnostic des applications proposées

Vous devez analyser l'ensemble des applications proposées dans la liste ci-dessous.

Pour chaque application, vous devez indiquer :

- les données principales manipulées ;
- les données temporaires ou rapides éventuelles ;
- les historiques éventuels ;
- les relations importantes entre entités ;
- les modèles NoSQL qui vous semblent les plus adaptés ;
- une justification courte de vos choix.

Cette première étape doit vous permettre de comparer les modèles NoSQL et de montrer que vous ne choisissez pas une base de données au hasard.

Étape 2 — Choix et développement d'une application

Après l'analyse, chaque groupe choisit **une seule application** parmi la liste.

Vous devrez développer une version simple de cette application en utilisant **au minimum deux modèles NoSQL différents**.

Le développement doit rester simple. L'objectif n'est pas de créer une application complète et professionnelle, mais de démontrer que vos choix de bases NoSQL sont cohérents avec les besoins fonctionnels.

Modèles NoSQL disponibles

Vous pouvez utiliser les modèles suivants :

Modèle NoSQL	Usage généralement pertinent
Document	Données principales, objets métier, fiches détaillées
Clé-valeur	Cache, compteur, score, session, état temporaire
Colonne	Historique, logs, données horodatées, événements répétés
Graphe	Relations entre entités, recommandations, réseaux, parcours

Vous devez choisir les modèles selon les besoins de votre application.

Applications proposées

Univers 1 — Coupe du monde de football

1. Application “Supporters et équipes favorites”

Description

L'application permet à des supporters de créer un profil, de choisir leurs équipes favorites et de suivre certains matchs.

Exemples de fonctionnalités possibles

- créer un supporter ;
- afficher la liste des équipes ;
- associer un supporter à une ou plusieurs équipes favorites ;
- associer un supporter à des matchs suivis ;
- afficher les supporters d'une équipe ;
- afficher les équipes suivies par un supporter ;
- afficher les matchs suivis par un supporter.

Points à analyser

Vous devez vous demander :

- quelles données doivent être stockées durablement ?
 - quelles relations sont importantes ?
 - faut-il représenter les liens entre supporters, équipes et matchs ?
 - certains compteurs ou données rapides peuvent-ils être utiles ?
-

2. Application “Réseau des joueurs et des équipes”

Description

L'application permet de visualiser les liens entre les joueurs, les équipes nationales, les postes et éventuellement les clubs.

Exemples de fonctionnalités possibles

- créer une fiche joueur ;
- créer une équipe ;
- associer un joueur à une équipe ;
- associer un joueur à un poste ;
- afficher tous les joueurs d'une équipe ;
- afficher tous les joueurs d'un même poste ;
- visualiser les relations entre joueurs, équipes et postes.

Points à analyser

Vous devez vous demander :

- quelles informations détaillées faut-il stocker sur les joueurs ?
 - quelles relations doivent être représentées ?
 - faut-il privilégier une structure orientée document, graphe ou les deux ?
 - comment justifier le choix du modèle relationnel le plus adapté ?
-

3. Application “Fan Votes”

Description

L'application permet aux supporters de voter pour un joueur, une équipe ou un homme du match.

Exemples de fonctionnalités possibles

- créer un vote lié à un match ;
- afficher les choix disponibles ;
- permettre à un utilisateur de voter ;
- afficher le nombre de votes par choix ;
- conserver l'historique des votes ;
- afficher le classement actuel des votes.

Points à analyser

Vous devez vous demander :

- où stocker la question du vote et les choix proposés ?
 - comment stocker chaque vote individuel ?
 - comment afficher rapidement les compteurs ?
 - faut-il conserver un historique horodaté ?
 - quels modèles NoSQL sont les plus adaptés pour ces besoins ?
-

4. Application “Live Match Tracker”

Description

L'application permet de suivre un match en direct avec des événements : but, carton, remplacement, début de mi-temps, fin de match, etc.

Exemples de fonctionnalités possibles

- créer un match ;
- ajouter les équipes participantes ;
- ajouter un événement pendant le match ;
- afficher la chronologie du match ;
- afficher le score actuel ;

- afficher uniquement les buts, les cartons ou les remplacements ;
- consulter l'historique complet d'un match.

Points à analyser

Vous devez vous demander :

- quelles données décrivent le match ?
 - quelles données évoluent en temps réel ?
 - quels événements doivent être historisés ?
 - quel modèle est adapté à une timeline chronologique ?
 - quel modèle est adapté à un score actuel ou un état temporaire ?
-

Univers 2 — Pokémon

Pour les sujets Pokémon, un fichier `pokemon.json` est fourni afin de vous permettre de vous concentrer sur les modèles NoSQL plutôt que sur la récupération des données depuis une API.

Vous pouvez importer ce fichier dans MongoDB ou l'utiliser comme base de données initiale.

5. Application “Pokédex personnel”

Description

L'application permet de consulter une liste de Pokémon et de créer un Pokédex personnel avec les Pokémon favoris d'un utilisateur.

Exemples de fonctionnalités possibles

- afficher la liste des Pokémon ;
- consulter la fiche détaillée d'un Pokémon ;
- rechercher un Pokémon par nom, type ou génération ;
- ajouter un Pokémon à son Pokédex personnel ;
- afficher les Pokémon favoris ;
- afficher les derniers Pokémon consultés ;
- afficher les Pokémon les plus populaires.

Points à analyser

Vous devez vous demander :

- quelles données Pokémon doivent être stockées ?
 - faut-il stocker les favoris d'un utilisateur ?
 - certaines recherches ou consultations peuvent-elles être mises en cache ?
 - quelles données sont durables et quelles données sont temporaires ?
-

6. Application “Team Builder Pokémon”

Description

L'application permet à un utilisateur de créer une équipe Pokémon composée de plusieurs Pokémon.

L'objectif est d'aider l'utilisateur à composer une équipe équilibrée selon les types, les rôles et éventuellement les faiblesses.

Exemples de fonctionnalités possibles

- afficher la liste des Pokémon disponibles ;
- ajouter un Pokémon à une équipe ;
- limiter une équipe à 6 Pokémon ;
- afficher les types présents dans l'équipe ;
- afficher les rôles des Pokémon ;
- visualiser les liens entre Pokémon, types et rôles ;
- identifier les faiblesses dominantes d'une équipe.

Points à analyser

Vous devez vous demander :

- quelles données doivent être stockées sous forme de documents ?
 - quelles relations sont importantes entre Pokémon, types et rôles ?
 - est-il pertinent d'utiliser un modèle graphe ?
 - certaines données peuvent-elles être stockées en cache ?
-

7. Application “Votes Pokémon favoris”

Description

L'application permet aux utilisateurs de voter pour leur Pokémon préféré et d'afficher un classement.

Exemples de fonctionnalités possibles

- afficher une sélection de Pokémon ;
- voter pour un Pokémon ;
- enregistrer chaque vote ;
- afficher le nombre de votes par Pokémon ;
- afficher le classement des Pokémon les plus populaires ;
- consulter l'historique des votes.

Points à analyser

Vous devez vous demander :

- où stocker les Pokémon disponibles au vote ?
- comment stocker les votes individuels ?

- comment afficher rapidement les compteurs ?
 - faut-il historiser les votes dans le temps ?
 - quels modèles NoSQL sont les plus adaptés ?
-

Travail attendu

1. Diagnostic comparatif

Avant de développer, vous devez produire un tableau de diagnostic pour les 7 applications proposées.

Pour chaque application, indiquez :

Application	Données principales	Données temporaires ou rapides	Historique éventuel	Relations importantes	Modèles NoSQL proposés	Justification
-------------	---------------------	--------------------------------	---------------------	-----------------------	------------------------	---------------

Ce tableau est obligatoire.

Il doit montrer que vous avez compris les différences entre les modèles NoSQL.

2. Choix d'une application

Chaque groupe choisit ensuite une application parmi les 7 proposées.

Vous devez expliquer pourquoi vous avez choisi cette application et quels modèles NoSQL vous allez utiliser.

Votre application doit utiliser **au minimum deux modèles NoSQL différents**.

3. Réalisation technique

Votre application doit proposer une version simple mais fonctionnelle.

Elle doit contenir au minimum :

- une base de données principale ;
- un deuxième modèle NoSQL utilisé pour un besoin différent ;
- un jeu de données de test ;
- des opérations simples de création, lecture ou mise à jour ;
- une démonstration claire de l'utilisation de chaque modèle NoSQL choisi.

Selon votre sujet, vous pouvez utiliser :

- des formulaires simples ;

- une interface web minimale ;
- une API Node.js / Express ;
- des scripts d'import ;
- MongoDB Compass ;
- Redis CLI ou RedisInsight ;
- Neo4j Browser ;
- Cassandra avec une interface ou des scripts simples.

L'interface graphique n'a pas besoin d'être avancée. Ce qui sera évalué en priorité est la cohérence entre le besoin applicatif et les modèles NoSQL choisis.

Contraintes minimales

Votre projet doit respecter les contraintes suivantes :

1. Utiliser au minimum deux modèles NoSQL différents.
 2. Justifier le rôle de chaque base utilisée.
 3. Insérer ou importer un jeu de données cohérent.
 4. Réaliser au moins trois opérations ou requêtes significatives.
 5. Présenter une architecture simple de votre application.
 6. Être capable d'expliquer pourquoi un autre modèle aurait été moins adapté.
-

Exemples de rôles possibles des bases NoSQL

Vous êtes libres de vos choix, mais voici des exemples de raisonnements attendus :

- une base document peut stocker des fiches détaillées ;
- une base clé-valeur peut stocker un compteur, un score ou un cache ;
- une base colonne peut stocker des événements horodatés ;
- une base graphe peut stocker des relations entre entités.

L'important n'est pas seulement d'utiliser plusieurs bases, mais d'expliquer pourquoi chacune est pertinente.

Livrables attendus

Chaque groupe devra rendre :

1. Un tableau de diagnostic

Le tableau doit analyser les 7 applications proposées et proposer les modèles NoSQL qui semblent les plus adaptés à chacune.

2. Une mini-application

L'application choisie doit être fonctionnelle, même de manière simple.

Elle doit démontrer l'utilisation d'au moins deux modèles NoSQL.

3. Un court document de justification

Ce document doit expliquer :

- le sujet choisi ;
- les fonctionnalités développées ;
- les modèles NoSQL utilisés ;
- le rôle de chaque base ;
- les collections, tables, clés ou relations créées ;
- les limites du projet ;
- les améliorations possibles.

4. Une démonstration orale

Chaque groupe devra présenter :

- le besoin de l'application ;
- l'architecture choisie ;
- les données stockées ;
- les requêtes ou opérations réalisées ;
- la justification des modèles NoSQL utilisés.

Barème indicatif

Critère	Points
Diagnostic des 7 applications	4
Pertinence du choix des modèles NoSQL	4
Réalisation technique de l'application	4
Utilisation correcte d'au moins deux modèles NoSQL	3
Données de test cohérentes	2
Justification technique	2
Présentation orale	1
Total	20

Conseils

Ne cherchez pas à développer une application trop complexe.

Un projet simple, clair et bien justifié sera mieux évalué qu'un projet ambitieux mais mal maîtrisé.

Vous devez surtout montrer que vous êtes capables de répondre à cette question :

Pour ce besoin précis, quel modèle NoSQL est le plus adapté, et pourquoi ?

Votre réflexion technique est aussi importante que le développement.